SHARDA UNIVERSITY

INVENTION DISCLOSURE FORM

CONFIDENTIAL

IMPORTANT

Please note that patent is always about a solution to an existing technical/industrial problem. As such, draft patent document (IDF) should be crafted in a manner so as to distinctly highlight the existing technical problem, the technical advancement proposed by the inventor(s), and the manner in which the proposed advancement solves the existing technical problem.

➢ **Particulars of Inventors**

S No	Name (Full)	Department / School	Designation	Mobile No.	Email	Official Address
1	Muhammad Shayaan Ali	Department of Computer Science and Engineering	Student	9136182311	2023433557.muhammad@ug.sharda.ac.in	Student,Greater Noida,Sharda University
2	Riddhima Sharma	Department of Computer Science and Engineering	Student	8860850003	2023567320.riddhima@ug.sharda.ac.in	Student,Greater Noida,Sharda University
3	Shruti Tyagi	Department of Computer Science and Engineering	Student	8826043366	2023566902.shruti@ug.sharda.ac.in	Student,Greater Noida,Sharda University
4	Aliza Rana	Department of Computer Science and Engineering	Assistant Professor	9818306962	aliza.rana@sharda.ac.in	Assistant professor,Greater Noida,Sharda University

Intelligent DevOps Monitoring & Incident Response System

Abstract :
The present invention is an Intelligent DevOps Monitoring and Incident Response System for continuously monitoring a software environment, detecting abnormal system behaviour, investigating operational incidents, and safely automating the response to them. The system collects infrastructure, container and application metrics with Prometheus. Abnormal behaviour is detected by two complementary detectors: configurable rule-based alerts for known failure modes, and a machine-learning anomaly detector (for example, Isolation Forest combined with rolling statistical scoring) for deviations that no predefined rule covers. Both detectors output events in a single, uniform incident format, and each event of interest is sent through a real-time WebSocket-based communication layer to an intelligent analysis component.

Before analysis, a context builder gathers the related metrics of every service in the dependency chain of the affected service over a time window around the event, so that the root-cause service can be distinguished from services that only show symptoms. The intelligent analysis component uses a Large Language Model (LLM) to interpret the incident and its context, determine the likely incident type and the likely root-cause service, and produce a structured, schema-validated response plan. The LLM never executes actions directly.

A policy validator placed between the LLM and the action executor checks every proposed action against an allowlist of actions, a list of known targets, a confidence threshold, cooldown rules and approval requirements before it can run; any action that is risky or has low confidence is sent to a human operator for approval. After execution, a verifier re-evaluates the triggering conditions; if the incident is not resolved, it is re-analysed together with the record of actions already tried, up to a configured maximum number of attempts, after which it is escalated to a human operator. When the LLM is unavailable, times out, returns invalid output or reports low confidence, deterministic runbooks provide a fallback response.

The system combines metric-based monitoring with statistical anomaly detection, dependency-aware incident analysis and safety-validated automated response, which reduces the need for continuous manual analysis of alerts. It provides a continuous monitoring-to-response pipeline of data collection, anomaly and alert detection, incident correlation and context building, real-time event delivery, intelligent incident analysis, policy validation, response execution and verification. This improves the efficiency and safety of DevOps operations compared with conventional monitoring systems, which mainly generate alerts without intelligent contextual analysis.

Field of Invention:

The present invention relates to DevOps, cloud infrastructure monitoring, observability of software systems, artificial intelligence, machine learning, and automated incident management. Specifically, the invention concerns an intelligent monitoring and incident-response mechanism that combines Prometheus-based metric collection, rule-based and machine-learning- based anomaly detection, dependency-aware context building across multiple services, real-time event communication over WebSocket, Large Language Model-based root-cause analysis, a policy-

based safety validation layer between the analysis engine and the action executor, and a verification and bounded re-analysis loop for detecting and handling operational problems in software and computing infrastructure.

Description Or Prior Art :

1) Conventional DevOps Monitoring Systems

Conventional DevOps monitoring tools continuously collect infrastructure and application metrics such as CPU usage, memory utilization, network activity, service availability, request rates and error rates. They can be configured to raise alerts when predefined thresholds or conditions are violated. However, such systems mainly offer observability and alert generation rather than complete, intelligent understanding and resolution of incidents. When an alert is raised, DevOps engineers often have to review several metrics, dashboards, logs and system components manually to establish the underlying cause, which increases the time needed to identify and resolve operational problems.

2) Rule-Based Alerting and Incident Management

Most monitoring tools use threshold-based or rule-based alerting, for example: CPU utilization above a predefined threshold, memory utilization above a predefined threshold, service availability below a set level, an error rate above a specified limit, or response latency above a specified value. While such rules are useful for detecting known conditions, they may not give enough contextual interpretation of complex incidents in which many metrics change at the same time. A single alert may also be linked to several underlying system conditions, which makes manual correlation necessary.

3) Conventional Incident Response

After an alert is raised, incident handling often requires a human operator to receive the alert, analyse monitoring information and logs, check related system behaviour, identify the likely cause, choose a suitable remediation, carry it out, and confirm that the issue has been resolved. This leaves a gap between incident detection and incident resolution: the monitoring system can show that something is wrong, but the reasoning and response that follow still depend on human intervention.

4) Limitations of Existing Systems

Existing approaches can suffer from one or more of the following drawbacks:

- Too many alerts to investigate manually.
- Difficulty correlating several monitoring indicators and log events.
- Limited contextual interpretation of alerts.
- Separation between monitoring, log analysis and incident-response capabilities.

- Reliance on fixed rules for known failure modes, with little ability to flag previously unseen anomalies.
- Longer time from alert notification to diagnosis.
- More human involvement in repetitive incident assessment.
- Difficulty turning raw monitoring data into an actionable response.
- No integrated pipeline connecting monitoring, anomaly detection, root-cause analysis and response execution.
- Threshold rules cannot pick up gradual or multi-metric deviations (for example, a slow memory rise) before a hard limit is reached.
- In multi-service systems, the alerting service is often only a downstream symptom, not the root cause.
- Applying LLM output directly to infrastructure is unsafe without validation, because LLMs can produce incorrect or hallucinated actions.

5) Gap and Motivation Identified

Considering the above limitations, there is a need for a system that connects continuous monitoring, statistical anomaly detection, dependency-aware incident correlation, real-time event processing, intelligent incident analysis and incident response in one integrated workflow. The gap is not simply a lack of monitoring data: modern monitoring systems can already collect large amounts of operational metrics and logs. The major gap is turning that information into context-aware understanding of the incident and an actionable, auditable response, while keeping a human operator in the loop for sensitive remediation actions. The present invention addresses this gap by introducing an intelligent processing layer between monitoring and incident response.

In particular, LLM reasoning must be applied to incident response in a way that guarantees that no unvalidated action reaches the infrastructure, and that remediation attempts are bounded and verified.

6) Existing AI-assisted Operations Tools

Recently launched tools such as k8sgpt, HolmesGPT and Datadog Bits AI use LLMs to troubleshoot infrastructure issues. These tools mainly describe or summarize issues, and either leave remediation to the engineer or perform actions without an independent, deterministic safety layer. They also do not combine rule-based and ML-based detection into a shared incident format with runbook fallback, or bound re-analysis using the record of actions already tried. Statistical anomaly detection methods (e.g., Isolation Forest, Liu et al., ICDM 2008) and graph-based root-cause methods (e.g., MicroRCA, Wu et al., NOMS 2020) detect or localize faults, but do not generate and safely execute a verified remediation.

7) Novelty of the Invention

Conventional monitoring tools (Prometheus, Datadog, Nagios) mainly detect problems and raise alerts. Newer AI-assisted operations tools (k8sgpt, HolmesGPT, Datadog Bits AI) use an LLM to explain or summarize an issue. They then either leave the fix to the engineer or act without an independent, deterministic safety layer. Closed-loop auto-remediation already exists, so the

novelty here is the combination: a monitoring-to-remediation loop in which an LLM proposes the action but never executes anything itself. Every action must pass a deterministic validator before it runs. The invention is characterized by:

1. Dependency-aware context with a settle interval. When the first event arrives, the gateway waits for a short, configurable settle interval. Events from the root-cause service and from services that only show symptoms are grouped into one incident. For every service in the dependency chain, the context builder collects metrics, container state, restart counts and recent actions, along with the service topology. This lets the LLM tell the root-cause service apart from the service that raised the alert.

2. An independent, deterministic policy validator between the LLM and the executor. Every response plan is treated as untrusted, whether it came from the LLM or from a runbook. An action runs automatically only if it passes all of these checks:
- schema validation
- the action allowlist
- the known-target check
- the confidence threshold
- the cooldown
- the per-incident action limit
- not being classed as risky

Risky or low-confidence actions go to a human for approval through the dashboard. The executor checks the allowlist again on its own, and arbitrary commands are never executed.

3. Pre-execution re-check. Just before an action runs, the triggering conditions are checked again. If the service has already recovered, the action is withheld.

4. History-aware, bounded re-analysis. If verification shows the incident is still unresolved, it goes back to the LLM with the attempt number and the actions already tried and their outcomes, so a failed action is not repeated. After a set maximum number of attempts (e.g., 3), the incident is escalated to a human operator.

5. Protection against stale plans. At most one action per incident can be in progress at a time (executing, being verified or awaiting approval). Any plan with an attempt number lower than the incident's current attempt is discarded, so a plan built on an outdated view of the incident can never trigger an action.

6. Validated fallback action. If the recommended action is rejected, the plan's fallback action is validated in its place. The incident is escalated only if neither action passes.

7. Two detectors, one incident format, deterministic fallback. Rule-based alerts and ML anomalies (Isolation Forest plus robust z-scores, with the anomaly required to persist for at least 2 consecutive windows) produce events in the same incident format. If the LLM times out, is unreachable, returns invalid output or reports low confidence, deterministic YAML runbooks produce the plan instead.

Results (prototype evaluation): The prototype ran on a three-service Docker Compose application. Nine fault scenarios were each injected 3 times under load into 3 configurations, for 81 runs in total.

- Every fault was detected and remediated automatically.
- The root-cause service was identified correctly in 81/81 runs.
- The executed action matched the expected action in 79/81 runs.
- No unsafe action was executed in any run.
- Adding the ML detector roughly halved the mean detection time, from 53 s to 27 s.
- The ML detector caught a gradual memory leak in about 45 s, compared with about 225 s for the threshold rule.
- In an adversarial test, none of 29 malformed, hallucinated or malicious response plans led to an unsafe action.

Summary of the Invention and Technical Advancement:

The invention provides the following technical advancements over existing systems:

1. Dependency-aware context builder. When an event is received, the system automatically collects the metrics, container states, restart counts and recent actions of all services in the dependency chain of the affected service over a time window around the event, together with the service topology. This enables the analysis engine to distinguish the root-cause service from services that only show the symptom.

2. Policy-based safety validator between the LLM and the executor. The LLM outputs a structured response plan that is validated against a schema. An action is executed only if it is on an action allowlist, its target is a known service, its confidence is at or above a configured threshold, it does not breach a cooldown period, and the per-incident action limit has not been reached. Otherwise it is sent for human approval or escalated. The LLM never executes commands directly.

3. History-aware, bounded re-analysis loop. If verification shows that the incident is not resolved, the incident is re-analysed with the attempt number and the list of actions already tried and their outcomes, so that a failed action is not repeated. After a maximum number of attempts (for example, 3), the incident is escalated to a human operator.

4. Dual detection with a unified incident format and deterministic fallback. Rule-based alerts and ML-detected anomalies are converted into a single common incident event. Deterministic runbooks provide a fallback response, so the system keeps working when the LLM fails, and they also act as a non-LLM baseline for comparison.

Disclose working of invention along with drawing, schematics and flow diagrams with explanation

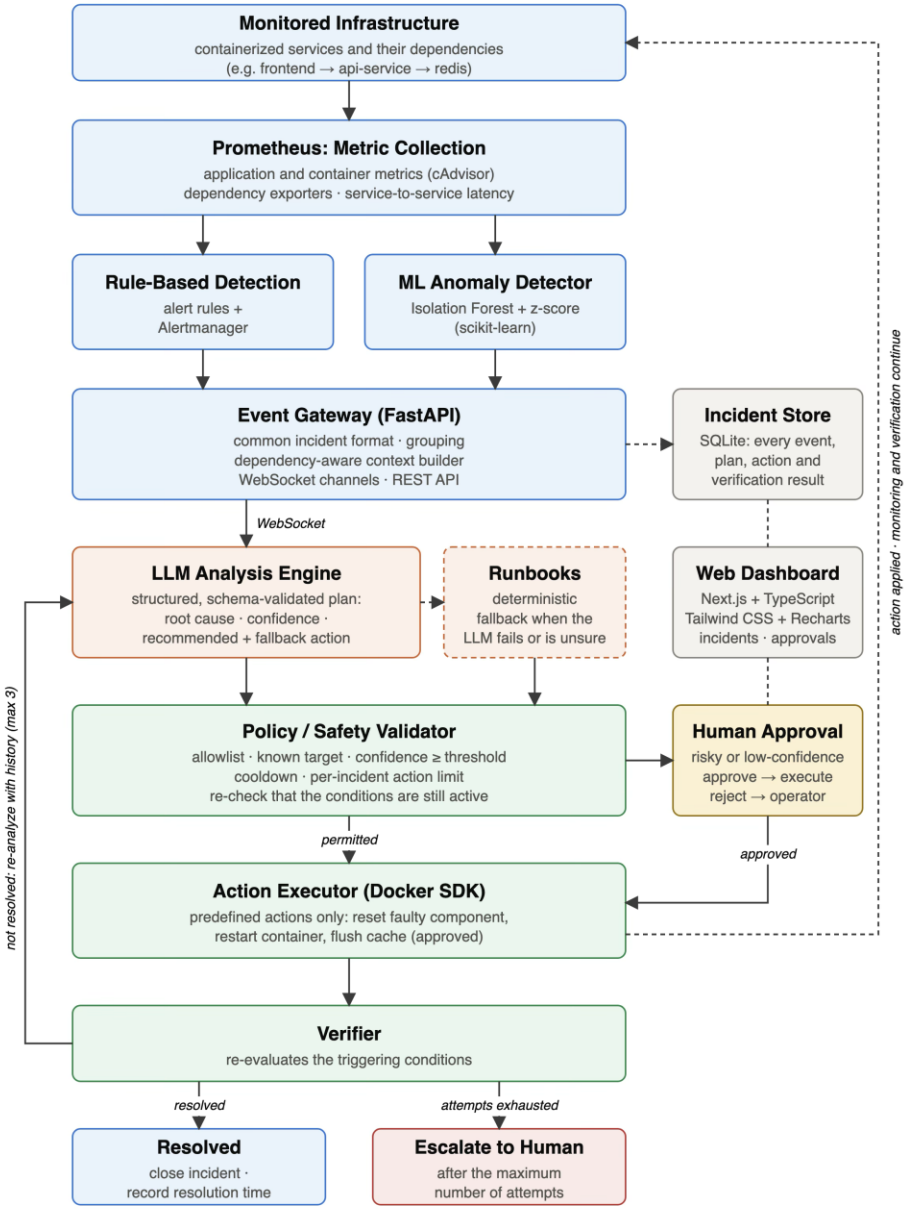


Fig 1. Proposed Intelligent DevOps Monitoring and Incident Response – System Architecture

The proposed system is divided into:

1) **Client Layer**

- Users access the platform through a monitoring and incident-response dashboard developed with Next.js and TypeScript

- The dashboard, styled with Tailwind CSS and visualized with Recharts, shows a service map with live metrics, current and past incidents with their root-cause analysis, the response plans and actions taken, the queue of actions awaiting human approval, and the health of every component of the pipeline.

2) **Gateway and API Layer**

- Monitoring events and dashboard requests are handled by an event gateway built with FastAPI (Python).

- The gateway receives rule-based alerts and ML anomalies, converts them into a common incident format, groups related events into one incident, builds the incident context, sends incidents and response plans over WebSocket channels, records every event, plan and action in the incident store, and exposes REST endpoints for the dashboard.

3) **Core Monitoring, Detection and Analysis Services**

- **Prometheus:** Continuously collects infrastructure and application metrics such as CPU, memory, service availability, request rate, error rate and response latency; container-level metrics through a container metrics exporter (cAdvisor); dependency metrics through exporters (e.g., for a Redis cache); and the latency of each call from one service to the next.
- **Rule-Based Detection:** Alert rules evaluated by Prometheus and routed by Alertmanager detect known failure conditions, such as high CPU or memory usage, high latency, a high error rate, or a service or dependency being down, and deliver them to the gateway.
- **Anomaly Detection Engine:** A Python and Scikit-learn based module analyses per-service features at fixed intervals with a model trained on normal behaviour, to detect deviations that no threshold rule covers, such as a gradual increase in memory usage.
- **Context Builder:** For each incident, gathers the last several minutes of the key metrics of every service in the dependency chain, together with container state, restart counts, the service topology and the actions already attempted.
- **Real-Time WebSocket Channels:** Send incidents to the analysis component, and response plans and action outcomes to the response system and the dashboard, with minimal delay.
- **LLM Analysis Engine:** Receives the incident and its context and performs incident identification, correlation, probable root-cause analysis and generation of a structured response plan. In the implemented prototype the LLM runs locally (an 8-billion-parameter model served by Ollama). Deterministic YAML runbooks provide the plan when the LLM fails or is not confident.
- **Response System:** A responder service consisting of a policy validator, an action executor using the Docker SDK, and a verifier. Only predefined actions, such as resetting a faulty component or restarting a container, are executed automatically; actions such as flushing a cache require human approval, and arbitrary commands are never executed.

4) **Data Layer**

- A relational incident store (SQLite in the prototype; PostgreSQL may be used at larger scale) records every incident, alert, response plan, executed action and verification result, and keeps an auditable history of the monitoring-to-resolution pipeline.

5) **Infrastructure**

- Docker and Docker Compose are used to run the complete system consistently across environments.
- A dockerized testing environment, seeded with intentionally faulty services, a fault injector that records the ground truth of every injected fault, and a load generator (Locust), is used to safely reproduce DevOps failure scenarios and validate the detection and remediation pipeline.
- Git and GitHub are used for source control and collaborative development.

6) **Overall Working**

Step 1 – Monitoring Data Collection. Prometheus continuously collects application, container and dependency metrics from the monitored infrastructure, including the latency of each call from one service to the next.

Step 2 – Detection (Rule-based and ML-based). Configured threshold rules evaluate the incoming metrics and raise alerts for known failure conditions. In parallel, the machine-learning anomaly detector samples per-service features (CPU usage, memory usage ratio, 95th-percentile latency, error rate, request rate and upstream call latency) at fixed intervals and scores them with a model trained on normal behaviour. It raises an event only when the anomaly persists for at least two consecutive evaluation windows. Each event carries the event source (rule or ML), the anomaly score where applicable, the top contributing features and an incident identifier.

Step 3 – Event Correlation and Context Building. Events that arrive within a time window and share a dependency chain are grouped into one incident. After the first event, the gateway waits for a short settle interval so that events from the root-cause service and from dependent services showing symptoms are analysed together. The context builder then attaches the service topology, the last N minutes of key metrics for every service in the chain, container state and restart counts, recent actions and the history of attempts. Ground-truth or fault-labelling signals, where present, are excluded from the analysis context.

Step 4 – Real-Time Event Transmission. The incident is sent through the WebSocket channel to the LLM Analysis Engine. Separate channels are used for incident events and for response plans and action outcomes.

Step 5 – Intelligent Incident Analysis. The prompt includes the service topology and the list of allowed actions. The LLM produces a structured response plan (root-cause service, incident type, severity, probable cause, evidence, confidence score, recommended action, fallback action and explanation), which is validated against a schema. If the LLM times out, is unreachable, produces output that fails validation, or reports a confidence below the threshold, deterministic runbooks produce the response plan instead. Safeguards against incorrect or hallucinated LLM output include: a limited set of actions supplied in the prompt; strict schema validation of the output;

rejection of unknown services or actions; a confidence threshold; required evidence fields that quote metrics from the supplied context; deterministic runbook fallback; and human approval for risky actions.

Step 6 – Decision (Policy Validation). A proposed action is executed automatically only if: (a) the response plan passes schema validation; (b) the action type is on the allowlist of predefined actions; (c) the target is a known service; (d) the confidence is a number between 0 and 1 at or above the threshold (e.g., 0.7); (e) the same action on the same target has not been taken within the cooldown period (e.g., 5 minutes); (f) the per-incident action limit (e.g., 3) has not been reached; and (g) the action is not classed as risky. Risky actions, such as flushing a cache or changing resource limits, and low-confidence plans are sent to a human operator for approval through the dashboard. If the recommended action is rejected, the plan's fallback action is validated in its place; if neither passes, the incident is escalated. The policy validator treats every response plan as untrusted input, regardless of its source.

Step 7 – Response Execution. Immediately before execution, the triggering conditions are checked again, and the action is withheld if they have already cleared. The executor carries out only predefined actions from the allowlist (for example, resetting a faulty component or restarting a container) and checks the allowlist again itself. Arbitrary shell commands are never executed.

Step 8 – Verification and Re-analysis. After a settling period (30–60 seconds), the verifier re-evaluates the conditions that fired for the root-cause service and the services that depend on it. If the incident is resolved, it is closed and its time to resolution is recorded. If it is not resolved, the incident is re-analysed with the attempt number increased by one and the list of previous actions and their outcomes, so that a failed action is not repeated. Only one action per incident is in progress at a time, and a plan produced for an earlier attempt is discarded. After the maximum number of attempts (e.g., 3), the incident is escalated to a human operator.

Step 9 – Continuous Monitoring. Prometheus continues monitoring the infrastructure, and every event, plan, action and verification result is recorded in the incident store and shown on the dashboard. The complete pipeline is: Prometheus → Rules + ML Detector → Gateway / Context Builder → WebSocket → LLM Analysis (with runbook fallback) → Policy Validator → Executor → Verifier.

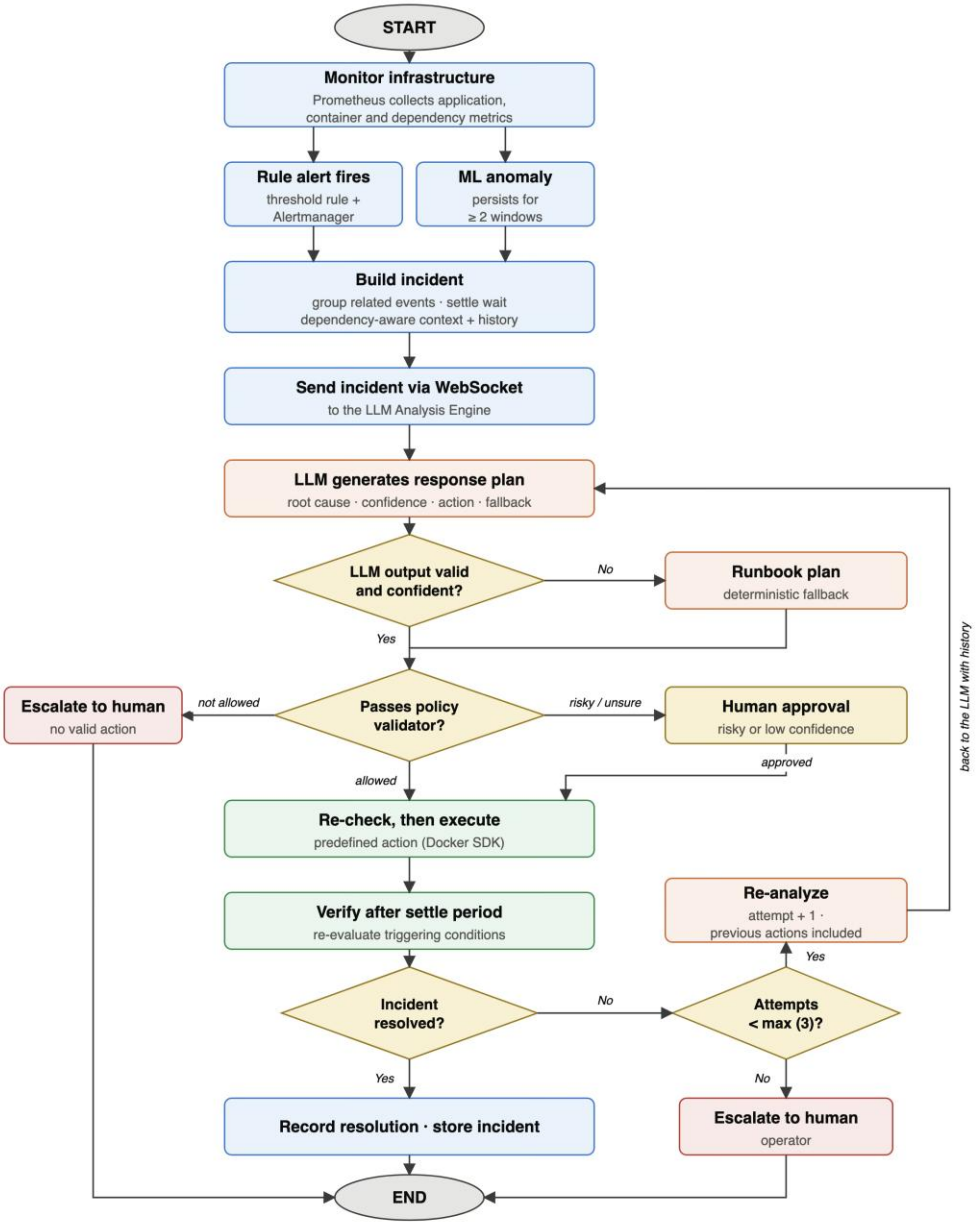


Fig 2. Workflow of Intelligent DevOps Monitoring and Incident Response

How it works:

The workflow begins with continuous monitoring of the DevOps infrastructure by Prometheus, which collects application, container and dependency metrics. Abnormal behaviour is identified either by a rule-based alert or by the Scikit-learn based anomaly detection model, and both produce an event in the same format. The gateway groups related events into one incident and builds its context from every service in the dependency chain, and the incident is sent through the WebSocket-based real-time communication layer to the LLM Analysis Engine. The LLM examines the context and produces a structured response plan; if the LLM fails or is not confident, a deterministic runbook produces the plan instead. The policy validator then checks the proposed action against the allowlist, the known targets, the confidence threshold, the cooldown and the per-

incident action limit; risky or low-confidence actions are sent to a human operator for approval through the dashboard. Approved actions are carried out by the Docker-SDK-based executor, and the verifier checks whether the incident has been resolved. If it has not, the incident is re-analysed together with the actions already tried, up to a maximum number of attempts, after which it is escalated to a human operator. All incidents, response plans, executed actions and verification results are stored in the incident store and shown on the dashboard.

GENERAL UTILITY / APPLICATIONS / ADVANTAGES OF THE INVENTION:

1. General Utility

The invention provides an intelligent mechanism for continuously monitoring software and computing infrastructure and for responding to the operational incidents that arise within it. Its main use is to bridge the gap between monitoring and alert generation on one side and incident resolution on the other, by combining metric collection, statistical anomaly detection, dependency-aware context building and LLM-based reasoning with a policy-validated, auditable remediation engine. The system is well suited to DevOps and Site Reliability Engineering (SRE) teams, helping them reduce alert fatigue, speed up root-cause identification, and respond to incidents faster, more safely and more consistently than with conventional monitoring systems.

2. Applications

- **Cloud Infrastructure:** Monitoring and responding to incidents in cloud-hosted computing environments.
- **DevOps and CI/CD Infrastructure:** Monitoring deployment environments, build systems, services and application infrastructure.
- **Containerized Environments:** Monitoring container-based applications and infrastructure, including Docker-orchestrated services.
- **Microservice Architectures:** Detecting and analysing operational issues in distributed services, including faults that propagate from one service to another, so that the alerting service differs from the root-cause service.
- **Web Applications:** Monitoring application availability, performance, errors and service behaviour.
- **Enterprise IT Infrastructure:** Supporting monitoring and incident response for enterprise servers, applications and services.
- **Production Systems:** Helping operational teams detect and respond to production incidents.

3) Advantages of Invention

- Continuous monitoring of infrastructure, container and application metrics.
- Statistical anomaly detection that works alongside conventional threshold-based alerting, detecting unknown or gradual anomalies before hard thresholds are crossed.

- Real-time event transmission between the monitoring infrastructure and the intelligent analysis system.
- Identification of the root-cause service, not only the service showing the symptom, using dependency-aware context.
- Intelligent, context-aware interpretation of alerts instead of relying only on raw monitoring information.
- No unvalidated LLM output reaches the infrastructure: actions must pass an allowlist, schema validation, a confidence threshold, a cooldown and, for risky actions, human approval.
- Automated or human-approved response to identified incidents.
- Bounded remediation with no infinite loops: a maximum number of attempts, then escalation.
- Continued operation when the LLM is unavailable, through deterministic runbook fallback.
- Less manual investigation effort for repetitive operational incidents.
- Faster transition from detection to response by integrating monitoring, analysis and remediation.
- Scalable, containerized architecture in which the monitoring, analysis and response components can be expanded independently.
- Human-in-the-loop capability, so that sensitive remediation actions can require operator approval.
- Continuous feedback, where post-response verification confirms whether the incident has been resolved.
- An auditable record of each incident's evidence, reasoning, action and verification result.

Implementation Status:

A working prototype has been built and tested end-to-end on Docker Compose with a three-service application (a frontend service, an API service and a Redis cache). It implements rule-based detection with Prometheus and Alertmanager, an event gateway with dependency-aware context building and WebSocket channels, an LLM analysis engine (a locally hosted 8-billion-parameter model) with deterministic runbook fallback, the policy validator, the action executor, the verifier with bounded re-analysis and escalation, an incident store, a machine-learning anomaly detector (per-service robust z-scores and an Isolation Forest, trained on metrics from normal operation), and a web dashboard built with Next.js, TypeScript, Tailwind CSS and Recharts. A fault injector reproduces failure scenarios and records their ground truth.

In a first evaluation, nine failure scenarios (CPU, memory, latency and error faults, stopped containers, a failed dependency, a gradual memory leak and a combined fault) were each injected three times under load into three configurations: rules with runbooks, rules with the LLM, and the complete system with the machine-learning detector (81 runs). Every fault was detected and remediated automatically, the root-cause service was identified correctly in every run, the executed action matched the expected action in 79 of 81 runs, and no unsafe action was executed. Adding the machine-learning detector halved the mean detection time (from 53 to 27 seconds) and detected the gradual memory leak in about 45 seconds, compared with about 225 seconds for the threshold rule. In an adversarial test, 29 malformed, hallucinated or malicious response plans resulted in no unsafe action being executed. The evaluation is being extended to 20 runs per scenario.

Claims:

1. A system for intelligent monitoring and incident response, comprising:

a metrics collection module configured to collect metrics of a plurality of services;

one or more detectors, including a rule-based detector, each configured to emit events in a common incident format;

an event gateway comprising a context builder configured to, on receipt of an event concerning a first service, retrieve metrics of the first service and of every service in its dependency chain over a time window associated with the event;

a real-time communication layer;

an analysis engine based on a large language model (LLM), configured to output a structured response plan comprising a root-cause service, a confidence score and a recommended action;

a policy validator, interposed between the analysis engine and an action executor, configured to permit execution only of actions satisfying an allowlist of action types, a known-target constraint, a confidence threshold and a cooldown constraint; and

a verifier configured to re-evaluate the triggering conditions after an action is executed.

2. The system of claim 1, wherein the one or more detectors further comprise a machine-learning anomaly detector configured to score per-service metrics with a model trained on normal behaviour, and wherein events from the rule-based detector and the machine-learning anomaly detector are converted into the common incident format.

3. The system of claim 1, wherein, when the verifier determines that the incident is unresolved, the incident is resubmitted to the analysis engine together with an attempt count and a record of previously attempted actions, and the incident is escalated to a human operator when the attempt count reaches a configured maximum.

4. The system of claim 1, further comprising deterministic runbooks applied when the analysis engine times out, is unreachable, returns output that fails schema validation, or returns a confidence below the threshold.

5. The system of claim 1, wherein actions classed as risky, and response plans whose confidence is below the threshold, are routed to a human approval interface instead of being executed automatically.

6. The system of claim 1, wherein fault-labelling or ground-truth signals are excluded from the context supplied to the analysis engine and to any detector.

7. The system of claim 2, wherein the machine-learning anomaly detector emits an event only after an anomaly persists for a configured number of consecutive evaluation windows, and the event includes an anomaly score and top contributing features.

8. The system of claim 1, wherein, immediately before executing a validated action, the triggering conditions are re-evaluated and the action is withheld if they are no longer active, so that an action is not applied to a service that has already recovered.

9. The system of claim 3, wherein at most one action per incident is in progress (executing, being verified or awaiting approval), and a response plan whose attempt number is lower than the incident's current attempt is discarded, so that a plan produced from an outdated view of the incident cannot trigger an action.

10. The system of claim 1, wherein the response plan further comprises a fallback action; when the recommended action is rejected by the allowlist, known-target or cooldown constraint, the fallback action is validated in its place, and when neither passes the incident is escalated.

11. The system of claim 1, wherein the policy validator operates independently of any validation of the analysis engine's output, treating every response plan as untrusted: non-finite or non-numeric confidence values, malformed action fields and malformed attempt identifiers are rejected, and the action executor independently re-checks the allowlist before executing.

12. The system of claim 1, wherein the event gateway, on receiving a first event, waits for a configured settle interval before forwarding the incident for analysis, so that events from the root-cause service and from dependent services showing symptoms are analyzed together as one incident.

13. The system of claim 1, wherein every event, response plan and action outcome is recorded in an incident store, from which detection time, root-cause accuracy, action correctness and time to resolution are computed against injected-fault ground truth.

14. A method for intelligent monitoring and incident response, comprising:

(a) collecting metrics of a plurality of services;

(b) detecting abnormal behaviour with one or more detectors, including a rule-based detector, and emitting an event in a common incident format;

(c) on receipt of an event concerning a first service, building a context comprising metrics of the first service and of every service in its dependency chain over a time window associated with the event;

(d) generating, with an LLM-based analysis engine, a structured response plan comprising a root-cause service, a confidence score and a recommended action;

(e) validating the recommended action, by a policy validator separate from the analysis engine, against an allowlist of action types, a known-target constraint, a confidence threshold and a cooldown constraint;

(f) executing the action only if it passes step (e);

(g) re-evaluating the triggering conditions after execution; and

(h) when the incident is unresolved, repeating steps (d) to (g) with an attempt count and a record of previously attempted actions, and escalating the incident to a human operator when the attempt count reaches a configured maximum.

15. The method of claim 14, wherein step (b) further comprises detecting abnormal behaviour with a machine-learning anomaly detector, and step (d) is performed by deterministic runbooks when the analysis engine fails, returns invalid output or returns a confidence below the threshold.

Disclaimer: The undersigned inventor(s) hereby declare that all statements, descriptions, and representations made in this Invention Disclosure Form are true, complete, and made to the best of their knowledge and belief. The inventor(s) shall bear sole responsibility and liability for the accuracy, validity, and authenticity of the information and data provided herein. The Research and Development Cell (RDC), the Institution, and any associated patent attorney, agent, or representative shall not be held liable or accountable for any inaccuracies, omissions, misrepresentations, or consequences arising from the information furnished by the inventor(s).

Signature of Inventor(s):

Signature: _____
Name: Muhammad Shayaan Ali
Date: _____

Signature: _____
Name: Shruti Tyagi
Date: _____

Signature: _____
Name: Riddhima Sharma
Date: _____

Signature: _____
Name: Aliza Rana
Date: _____